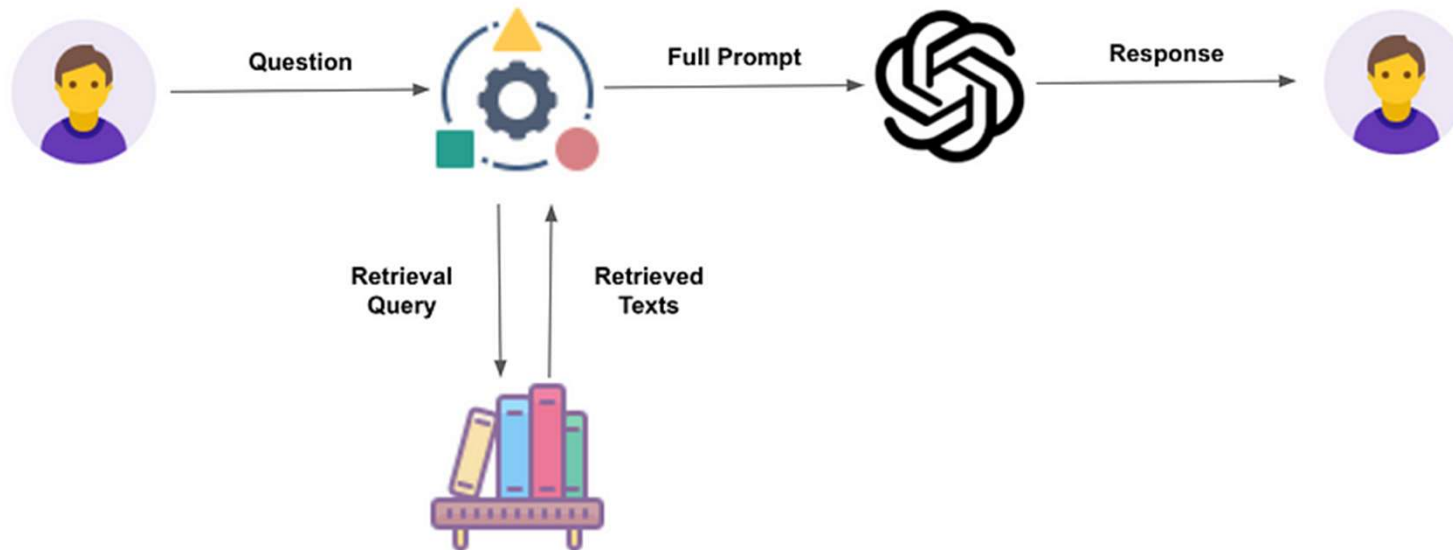


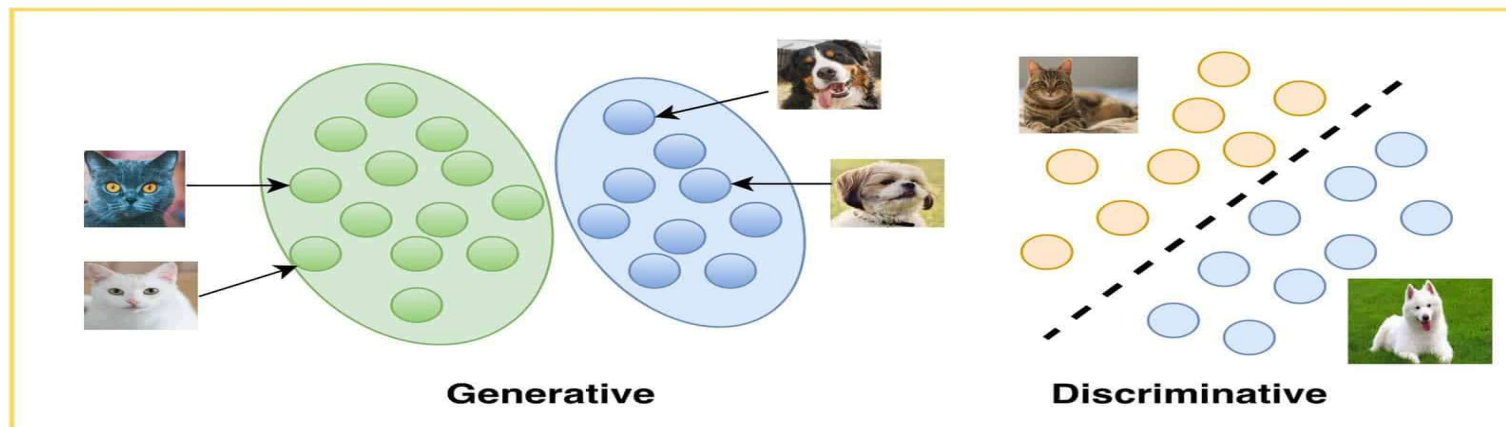
Welcome to World of LLM's



By Varun Raste ,
Data Scientist - IBM

Retrieval Augmented Generation





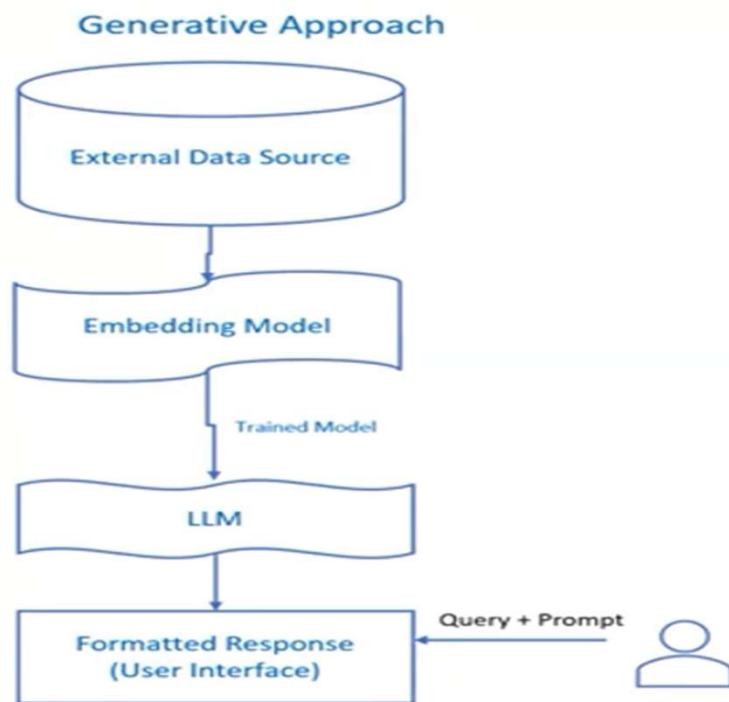
Generative:

- Learns the underlying data distribution, allowing for new data creation (e.g., image synthesis, text generation).
- Can be useful for tasks like anomaly detection and data augmentation.
- Often more complex and computationally expensive to train.
- Can be used for applications like data compression and dimensionality reduction.

Discriminative:

- Focuses on predicting labels or categories for new data points (e.g., image classification, spam detection).
- Generally faster to train and more efficient for specific prediction tasks.
- May not be able to explain the reasoning behind their predictions.
- Widely used for tasks like classification, regression, and recommendation systems.

Generative Approach

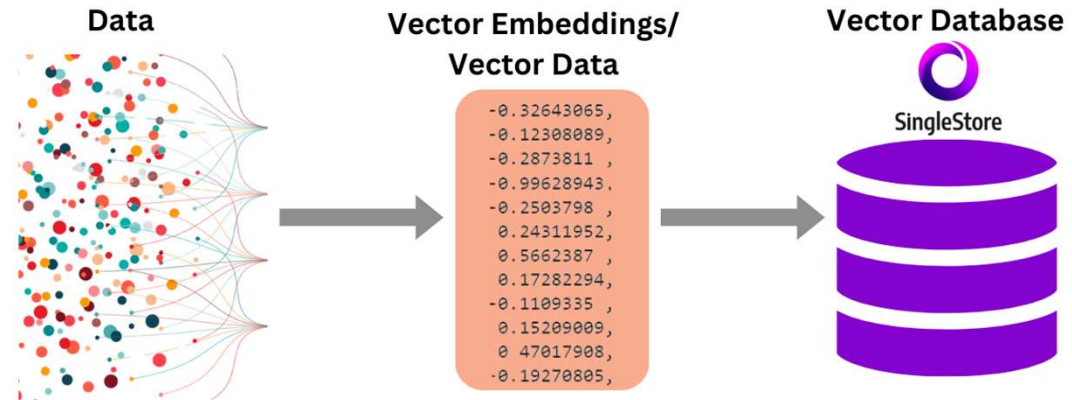


Source: www.ashutoshtripathi.com

- Traditional Method
- Works by training the LLM
- Training the entire LLM is a tedious process
- Runs out of latest developments

Vector Store & Database

- A vector store is a lightweight system whose primary responsibility is to store embeddings and perform similarity search efficiently.
- A vector database is a full-fledged database system designed to store, index, manage, and query vectors at scale, often with metadata filtering, persistence, and fault tolerance.
- Vector stores are typically used inside applications or libraries, while vector databases are used as infrastructure components.
- The difference is less about math and more about engineering maturity and production readiness.



Vector Store vs Vector Database

Aspect	Vector Store	Vector Database
Primary Purpose	Fast similarity search	Scalable vector management
Scope	Embeddings + search only	Storage, indexing, filtering, scaling
Deployment	In-memory or local	Distributed, production-grade
Metadata Support	Limited or basic	Rich filtering & indexing
Persistence	Optional	Strong persistence guarantees
Use Case	Prototypes, demos, small apps	Production search & RAG systems

Comparison Summary

Vector Store	Type	Strengths	Limitations	Best Fit Use Case
FAISS	Library (in-memory / disk)	Extremely fast similarity search, highly optimized ANN algorithms, full control over indexing	No built-in metadata filtering, no native distributed support, manual persistence	Research, prototyping, offline batch search
Annoy	Library (file-based)	Simple, lightweight, memory-efficient, good for read-heavy workloads	Static index (no frequent updates), limited filtering	Recommendation systems with static embeddings
Chroma	Embedded vector store	Very easy to use, integrates well with LLM frameworks, supports metadata	Limited scalability, not built for heavy production load	Local RAG apps, demos, teaching
Milvus	Distributed vector database	Horizontally scalable, supports large datasets, strong ANN performance	Requires infra setup, operational overhead	Large-scale enterprise search, production RAG
Weaviate	Vector database + API	Rich metadata filtering, schema support, hybrid search	Higher memory usage, tuning required	Semantic search with structured filtering
Pinecone	Fully managed cloud service	No infra management, auto-scaling, high availability	Cost, vendor lock-in, limited internal control	Fast production deployment, startups
Qdrant	Vector database	Strong filtering, efficient HNSW indexing, open-source friendly	Smaller ecosystem than cloud-managed options	Production RAG with strong control needs

Information Retrieval

Sparse Retrieval

Focuses on: Representing documents and queries with vectors that are mostly zeros.

Benefits:

- Faster processing and less storage needed.
- Easier to understand why documents are retrieved (focus on non-zero terms).
- Compare to: Dense retrieval (many non-zero values, captures more detail but less efficient).
- Examples: Bag-of-Words (BoW)
- **Key method: BM25:** Assigns weights to terms based on factors like frequency in the document and rarity across the collection. Only terms in the document or query will have non-zero weights in the final vector.

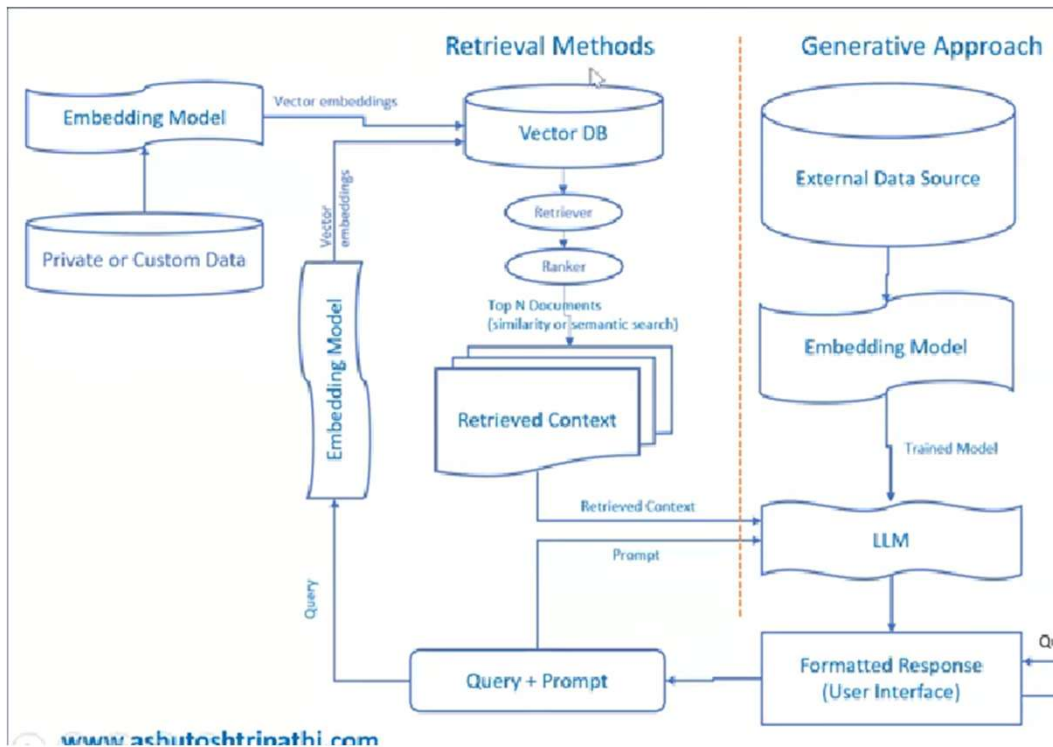
Dense Retrieval

Utilizes high-dimensional vectors to represent documents and queries.

Benefits:

- Dense vectors can capture more intricate relationships between words and concepts compared to sparse methods.
- The higher dimensionality allows for more nuanced representation, potentially leading to better retrieval performance.
- Examples : Deep Learning Encoders: Large language models like BERT or Transformer models are used to generate dense vector representations for documents and queries.

RAG Approach

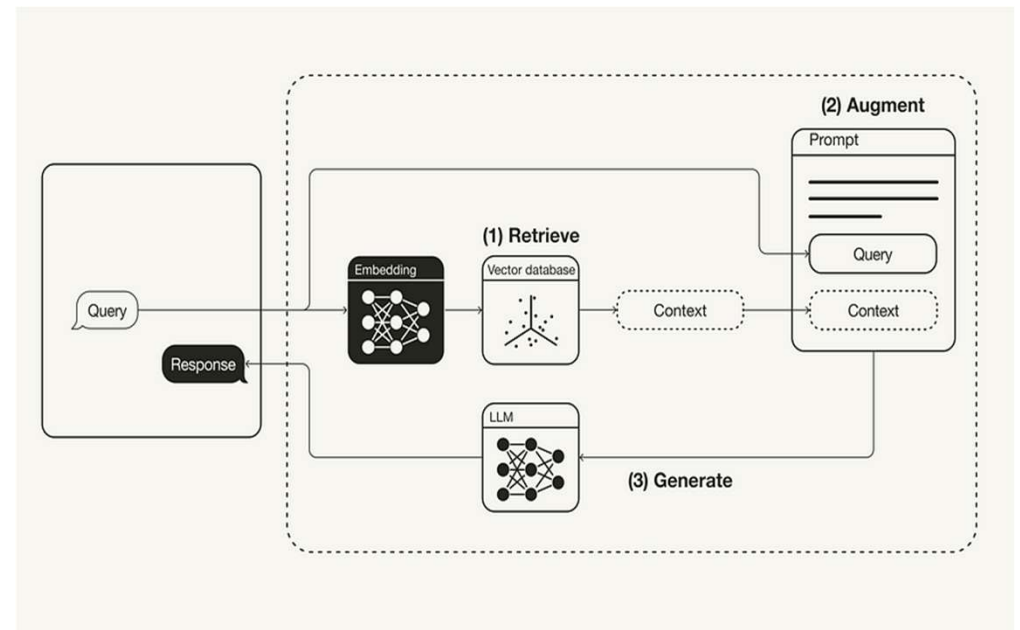


Source: www.ashutoshtripathi.com

- Makes use of external resources
- Components of RAG :
Retriever/ Searcher, Ranker
- Instead of training the entire LLM , it just appends embeddingd of external data sources to existing LLM.

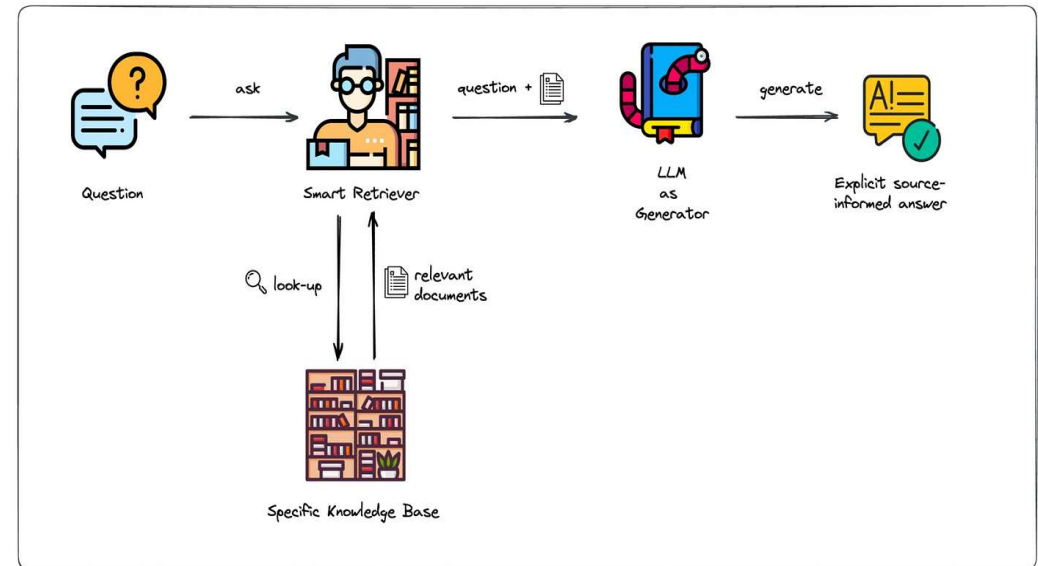
Basic Flow

- Indexing: Prepare the knowledge base by creating summaries and assigning unique identifiers to each piece of information.
- Retrieval: Analyze user queries and find relevant information from the indexed knowledge base.
- Augment: User adds a prompt chain for LLM in order to go for query generation from chosen context
- Generation: Use the retrieved information and the user query to guide a large language model in generating an accurate response.



Retriever

- Finds relevant info for the LLM in a massive knowledge base.
- Analyzes user queries to understand their intent and information needs.
- Scans indexed documents to find matches based on the analyzed query.
- Ranks retrieved documents based on relevance, returning the most fitting ones.
- Can be Dense (powerful, deep learning for complex meaning) or Sparse (faster, focuses on key terms).
- Choosing the retriever depends on data size, accuracy needs, and computing power.

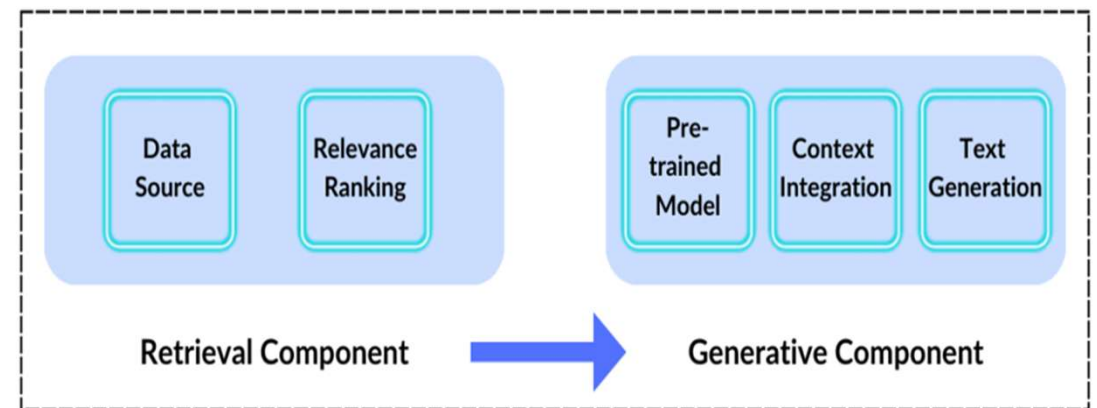


Generative

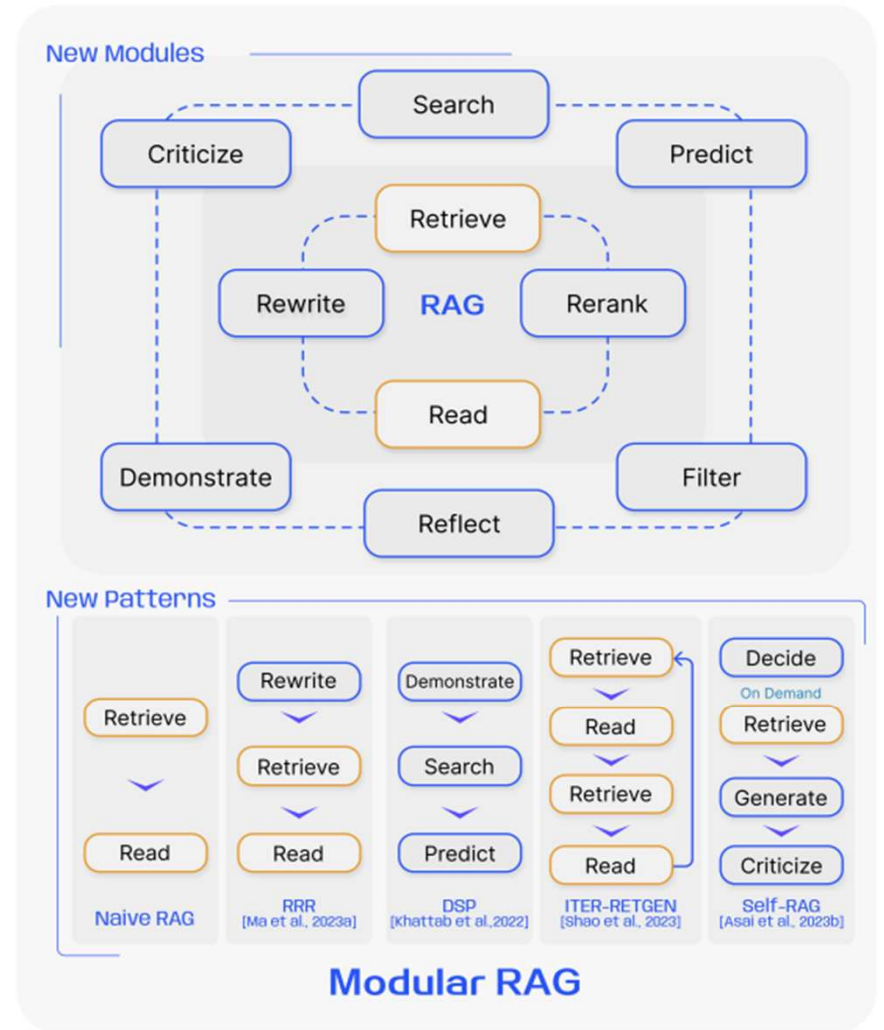
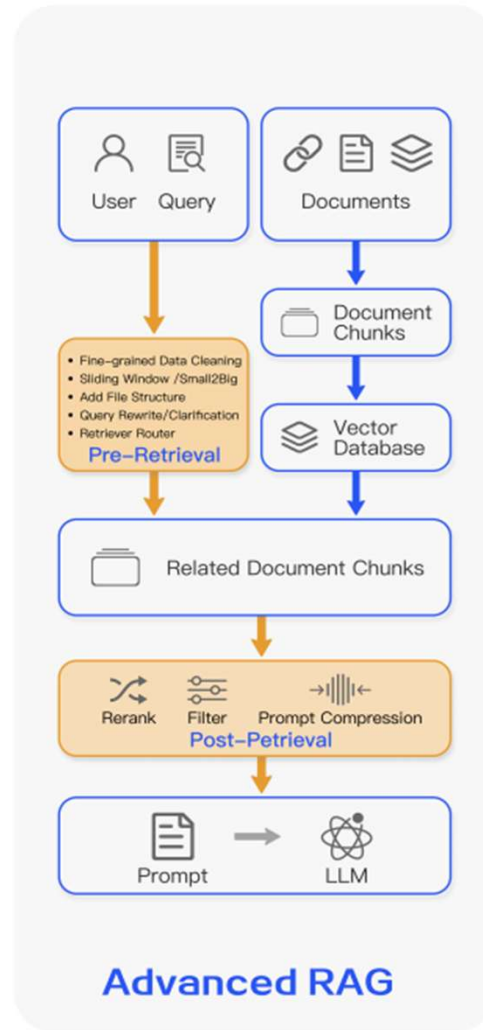
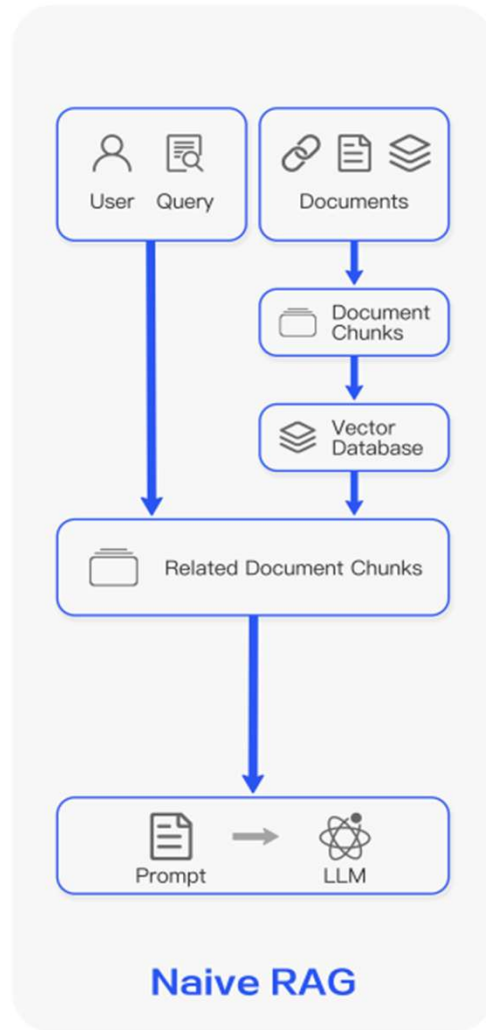
Uses retrieved info and the query to craft a comprehensive response.

- Analyzes key points and context from retrieved information.
- Formulates a coherent response that addresses the user's needs.
- May personalize the response based on intent and context.
- Large Language Model (LLM): Powerful engine for human-quality text generation.
- Text Summarization: Condenses information for concise responses.
- Overall: Transforms retrieved knowledge into a user-friendly and informative response.

Retrieval-Augmented Generation (RAG) System



Types of RAG



Naive RAG

Strengths:

- Simple and Efficient: Easy to implement and computationally efficient due to the separation of retrieval and generation tasks.
- Effective for Focused Queries: Delivers good results for well-defined queries with a clear information need.
- Flexible Knowledge Base: Can handle various information sources (text, code, data) within the knowledge base.
- Improved Explainability: Retrieved information can potentially provide some level of explainability for the generated response.

Weaknesses:

- Prone to missing relevant information, especially for complex or ambiguous queries.
- Relies heavily on the quality of indexing within the knowledge base.
- May lead to responses that lack coherence or depth.
- Potential for Inaccurate Responses .Retrieved information might not perfectly align with the user's intent, resulting in inaccurate or misleading responses.
- Limited Control over Generation, Offers less control over the style or tone of the generated response compared to more advanced RAG models.

Advanced RAG

Strengths:

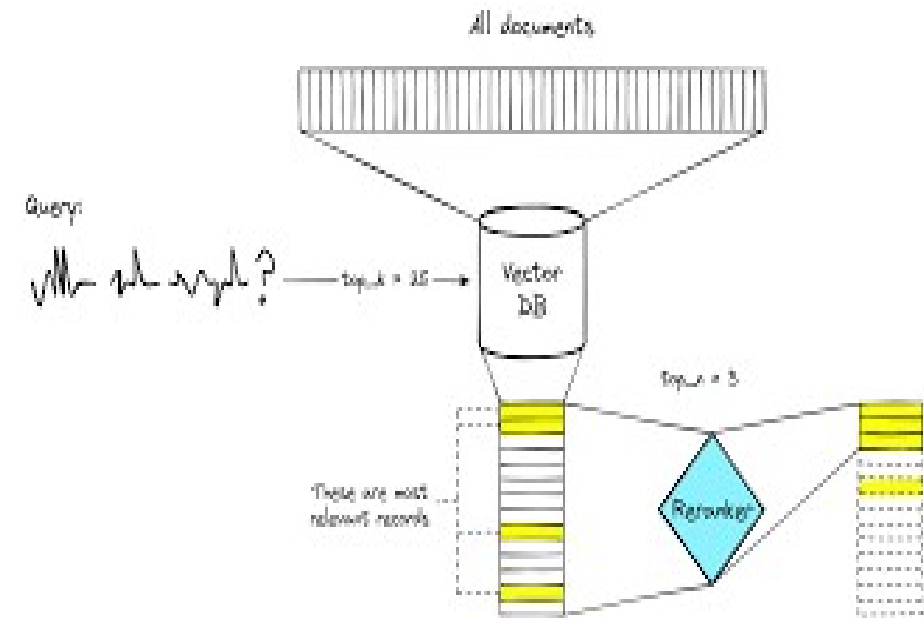
- **Improved Retrieval:** Techniques like better ranking and multi-source retrieval enhance the accuracy and completeness of retrieved information.
- **Enhanced Context Integration:** Advanced RAG models can better integrate information from multiple sources, leading to more coherent and informative responses.
- **Conditional Generation:** Fine-tuning response generation based on the query and context allows for more targeted and accurate responses.
- **Greater Control:** Some advanced models offer more control over the style and tone of the generated text.

Weaknesses:

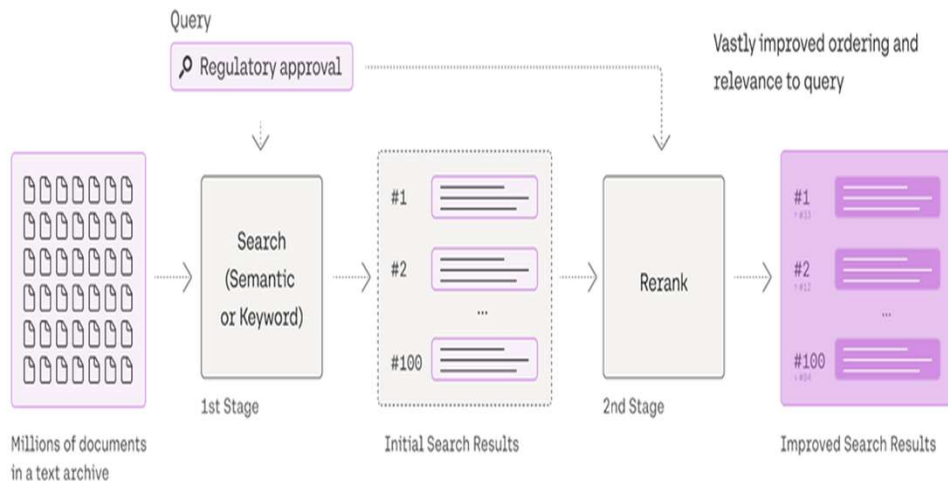
- **Increased Complexity:** These models can be more computationally expensive to train and run due to their sophistication.
- **Potential for Overfitting:** Advanced models trained on specific data might not generalize well to unseen information.
- **Reliance on Training Data:** The effectiveness of advanced RAG heavily depends on the quality and quantity of training data

Advanced RAG - Rerankers

- **Improved Relevance:** Advanced RAGs use deep learning models like BERT to prioritize the most relevant documents.
- **Enhanced Context Understanding:** Transformer models capture nuanced contextual relationships for accurate document ranking.
- **Noise Reduction:** Sophisticated rerankers filter out irrelevant documents, providing high-quality input for generation.
- **Handling Complex Queries:** Advanced rerankers effectively manage complex queries with enhanced modeling capabilities.
- **Two-Stage and Multi-Pass Reranking:** Initial broad retrieval followed by refined reranking improves retrieval quality (e.g., using BERT for re-ranking after BM25 retrieval).

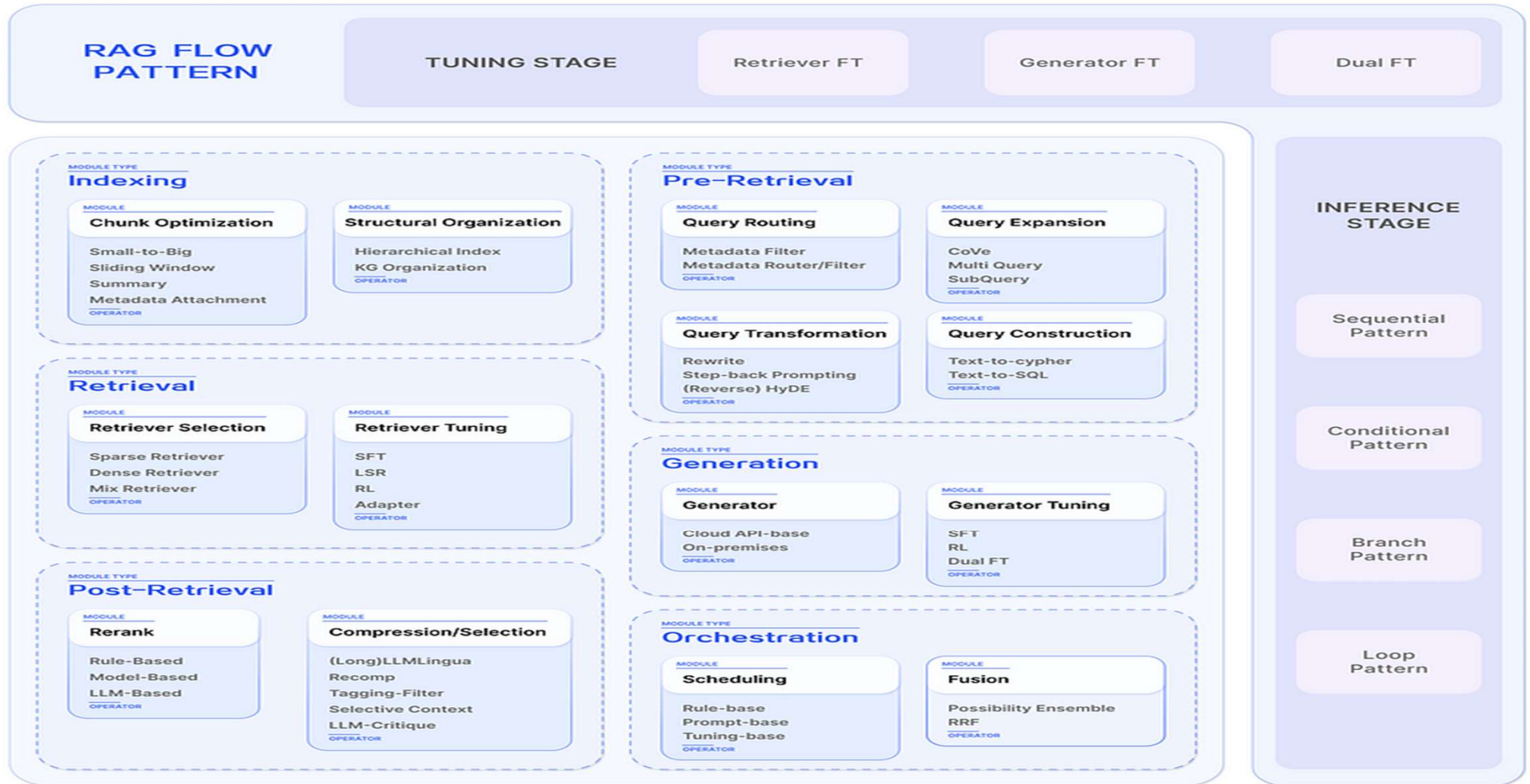


Cohere Re-ranker

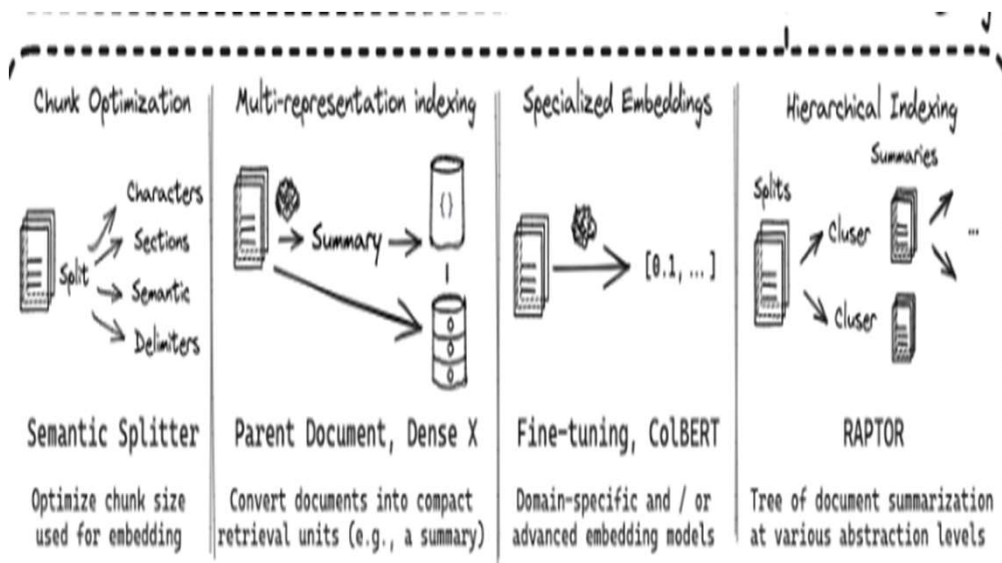


- Cohere Reranker takes a search query and a list of documents as input.
- It uses a large language model to compute a relevance score for each query-document pair.
- This scoring considers semantics, meaning it goes beyond just keywords to understand the meaning of the query and document.
- Documents are then re-ranked based on their relevance scores, with the most relevant documents appearing at the top of the search results.
- Cohere Reranker can be fine-tuned on domain-specific data to improve performance for specialized searches.

Modular RAG



Indexing



- **Chunk Optimization:** Breaking down content into smaller, more manageable units for faster indexing and retrieval.eg: small to Big , Sliding Window
- **Semantic Splitting:** Dividing content based on meaning, like splitting paragraphs into sections based on topic shifts, for improved semantic search.
- **Multi-representation Indexing:** Creating multiple indexes for the same content using different representations (e.g., keywords, phrases, semantic embeddings) to facilitate various search queries.
- **Specialized Embeddings:** Training custom embeddings specific to the domain or data type for a deeper understanding of content during indexing and retrieval.
- **Hierarchical Indexing:** Organizing content into a hierarchy (like categories and subcategories) to enable efficient browsing and faceted search

Pre-Retrieval Stage

Challenges in Retrieval

- Poorly worded queries
- Language complexity & ambiguity (e.g., LLM meaning)

Query Expansion

- Enriching the query with additional context for better retrieval
- **Multi-Query:** Expanding the single query into multiple, diverse, and relevant sub-queries using prompt engineering with LLMs. A challenge is maintaining user intent - mitigate by giving more weight to the original query.
- **Sub-Query:** Decomposing a complex question into simpler sub-questions using least-to-most prompting.
- **CoVe (Chain-of-Verification):** Validating expanded queries using LLMs to reduce retrieval hallucinations.

Query Transformation

- Using a transformed query instead of the original for retrieval
- **Rewrite:** Prompting LLMs or using specialized models (e.g., RRR) to rewrite the original query for better retrieval. (e.g., BEQUE system improves recall for long-tail queries)

HyDE (Hypothetical Document Embeddings):

- LLM focuses on retrieving answer-to-answer similarity instead of query-to-answer. (Also includes Reverse HyDE for query-to-query retrieval).
- **Step-back Prompting:** Abstracting the original query to a high-level concept question for retrieval alongside the original query.

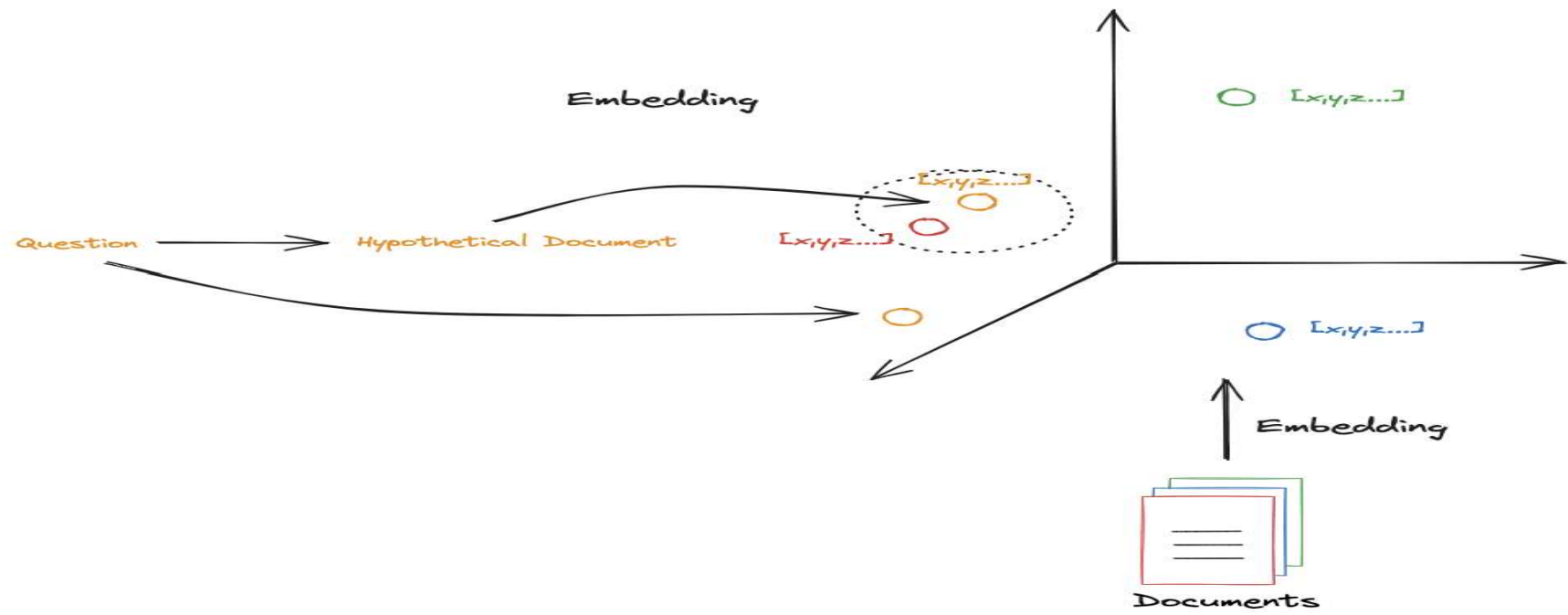
Query Routing

- Routing queries to different RAG pipelines based on their nature for a versatile system.
- **Metadata Router/Filter:** Filtering chunks based on keywords extracted from the query and metadata within the chunks.
- **Semantic Router:** Routing based on the semantic information of the query. (Hybrid approaches combining both methods are also used.)

Query Construction

- Converting user queries into a different query language to access alternative data sources.
 - Text-to-Cypher
 - Text-to-SQL
- Combining structured query languages (SQL, Cypher) with semantic information and metadata for complex queries.

HYDE



Retrieval

1. Importance of Retrieval:

- Powerful tool for finding relevant information for RAG.
- Uses latent spaces to compare queries and documents.

2. Key Considerations:

- **Efficiency:** How fast can information be retrieved?
- **Embedding Quality:** How well are queries and documents represented?
- **Alignment:** Do models, data, and tasks work together effectively?

3. Retriever Selection:

- No single "best" model, depends on your needs.
- **Sparse Retrievers:** Efficient, good for initial filtering (e.g., BM25, TF-IDF).
- **Dense Retrievers:** More powerful, use neural networks (e.g., ColBERT, BGE).
- **Mix/Hybrid:** Combine different approaches for better results.

4. Retriever Fine-tuning:

- Improve retrieval for specific domains (e.g., healthcare, law).
- Several techniques: SFT, LSR, RL, Adapters.

Post Retrieval

Challenges of Raw Retrieval:

- **Lost in the Middle:** LLMs struggle to remember the full context, especially from lengthy passages.
- **Noise/Anti-Facts:** Irrelevant or contradictory information can lead to inaccurate outputs.
- **Context Window:** Limited processing capacity restricts the amount of information LLMs can effectively use.

Reranking Techniques:

- **Rule-Based Reranking:** Uses metrics like relevance, diversity, and MMR (Maximal Marginal Relevance) to prioritize document chunks.
- **Model-Based Reranking:** Employs another language model (e.g., BERT, Cohere rerank) to reorder retrieved chunks based on importance.

Compression and Selection Techniques:

- **Compression:** Aims to reduce the amount of information fed to the LLM while preserving key points. Approaches include:
 - **Small LLM Compression:** Trained models like GPT-2 Small identify and remove unimportant parts, creating a compressed prompt.
 - **Recomp:** Offers two compressor types:
 - **Extractive:** Selects relevant sentences.
 - **Abstractive:** Generates summaries from multiple documents.
 - **Selective Context:** Similar to "stop-word removal," identifies and removes redundant content based on self-information.

Other Techniques:

- **Tagging-Filter:** Assigns labels to documents and filters based on the user's query.
- **LLM-Critique:** The LLM itself evaluates retrieved content and filters out irrelevant documents.

Generation

Generator Selection:

- **Cloud API-Based Generators:** Utilize external LLMs through APIs (e.g., OpenAI's ChatGPT).

Benefits: Easy setup, access to powerful models.

Drawbacks: Data privacy concerns, limited control over models.

- **On-Premises Generators:** Locally deployed open-source or custom LLMs.

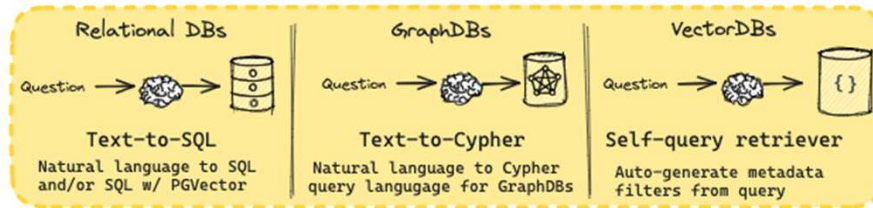
Benefits: More control, better data privacy.

Drawbacks: Requires computational resources, limited model options.

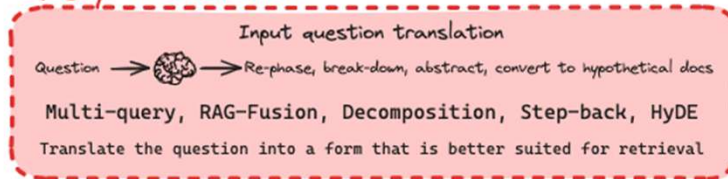
Generator Fine-Tuning:

- Improves LLM performance by tailoring it to specific tasks or domains.
- Common methods include:
 - **Supervised Fine-Tuning (SFT):** Adds domain-specific knowledge to the LLM.
 - **Reinforcement Learning (RL):** Trains the LLM to generate outputs aligned with human or retriever preferences.
 - **Distillation:** Creates a smaller, more efficient model from a larger, more powerful one.
- **Dual Fine-Tuning:** Aligns the preferences of both the retriever (finding information) and the generator (using information) for better overall performance.

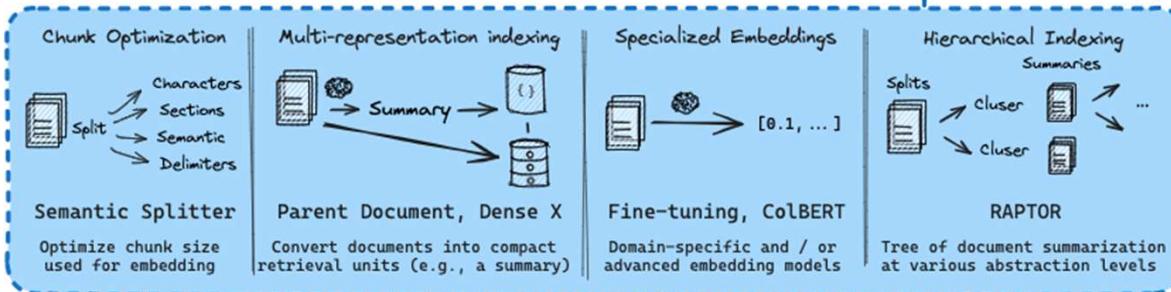
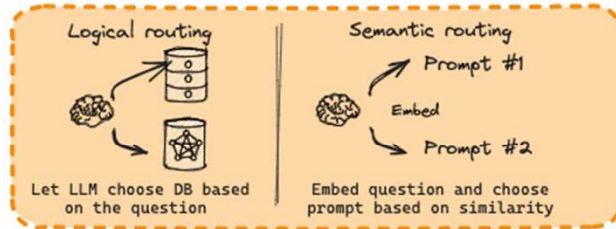
Query Construction



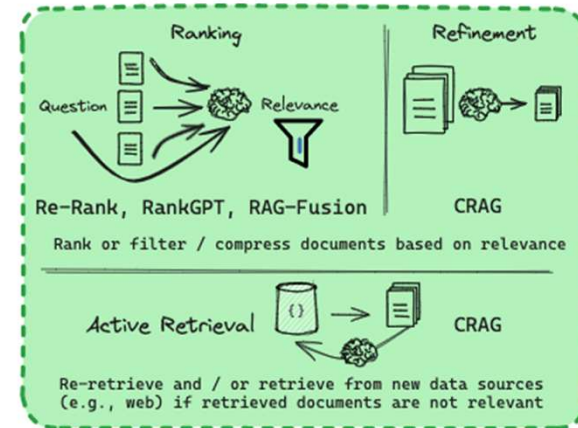
Query Translation



Routing

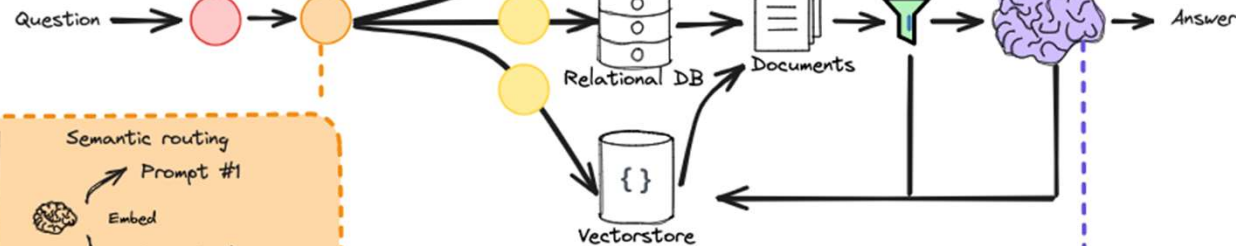


Retrieval

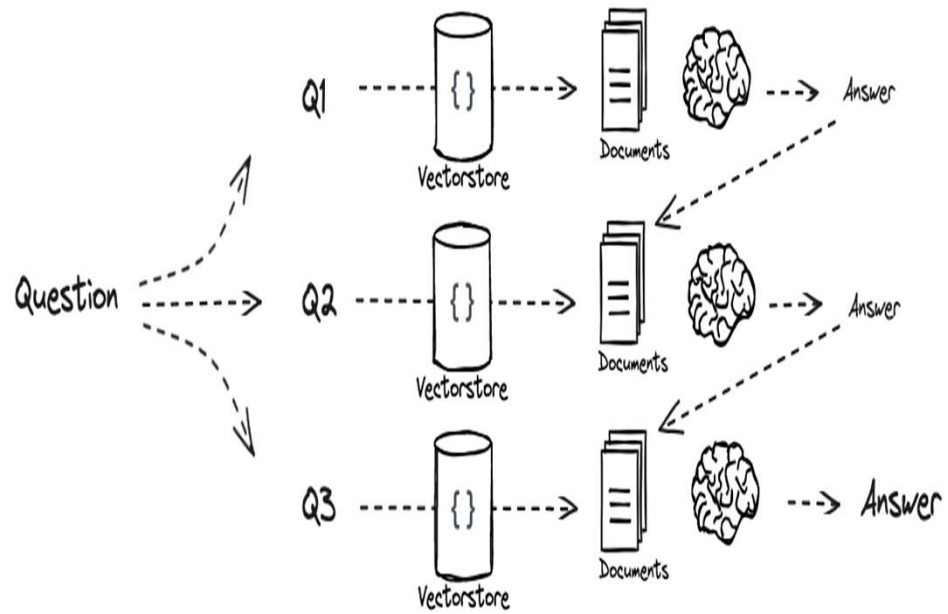


Indexing

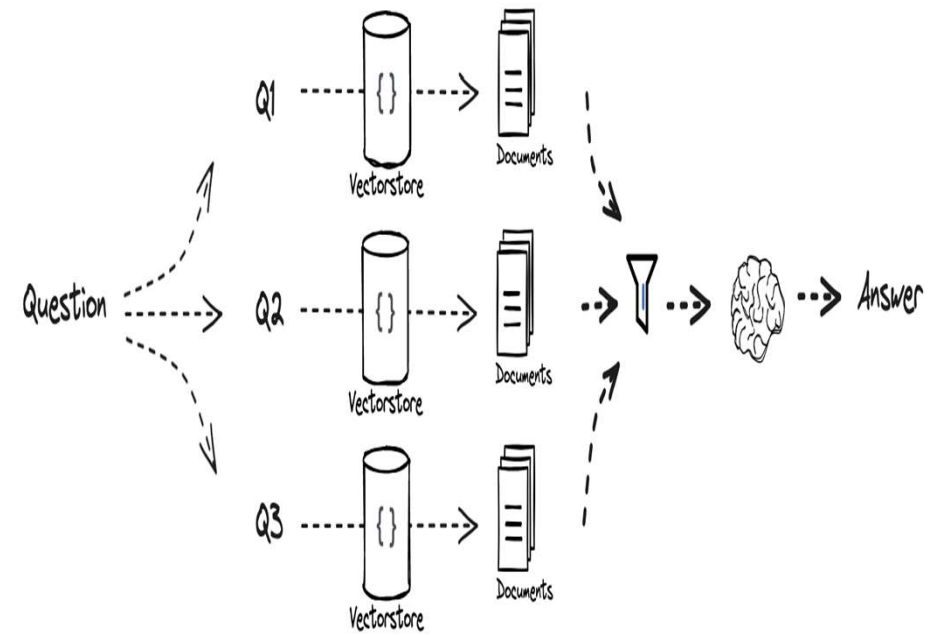
Generation



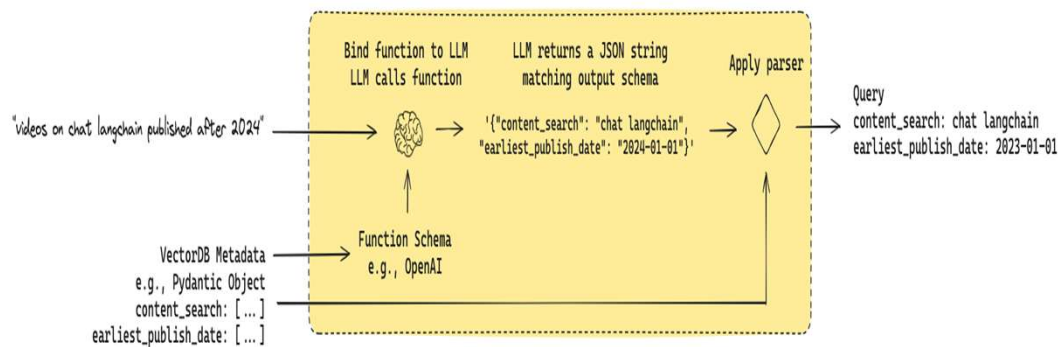
Decomposition



RAG Fusion

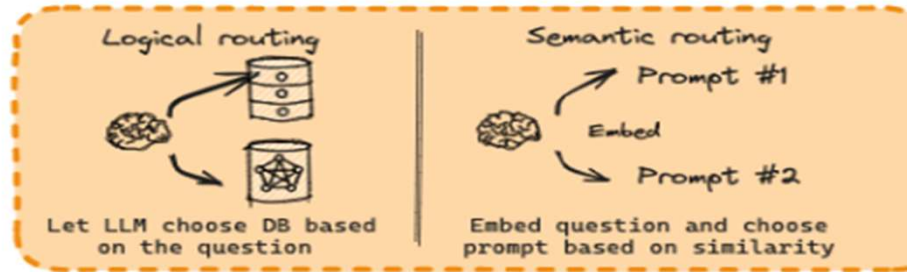


Query structuring for metadata filters



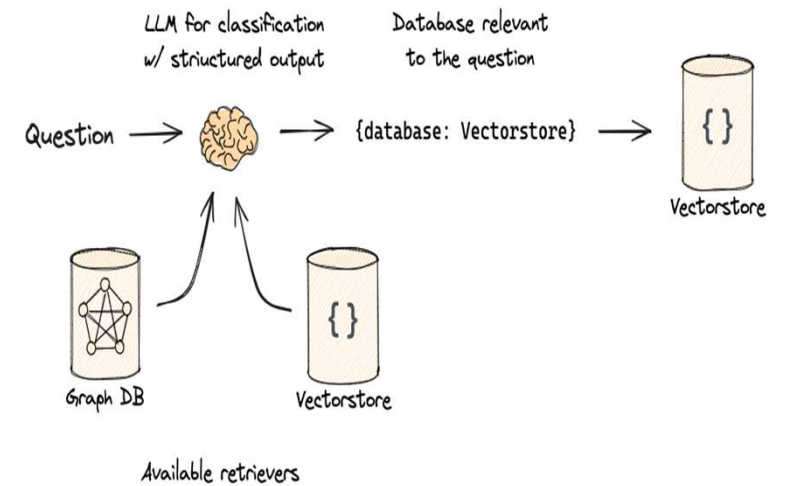
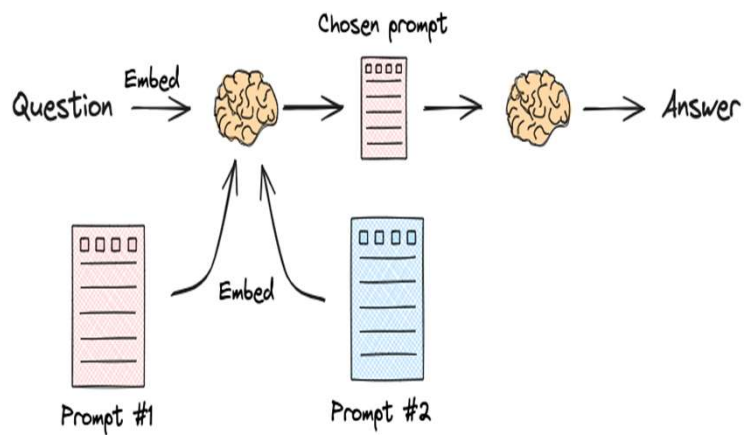
- Targeted Search: Metadata filters allow you to refine your search by specifying additional criteria beyond keywords.
- Focus on Specifics: You can filter based on metadata fields like creation date, author, category, or custom tags.
- Structured Queries: The query structure typically involves specifying the metadata field, operator (e.g., equals, greater than), and desired value.
- Improved Accuracy: Filtering by metadata helps narrow down search results and retrieve the most relevant items.
- Varied Applications: Metadata filter syntax can differ between systems, but the core concept remains the same across search platforms and databases.

Routing



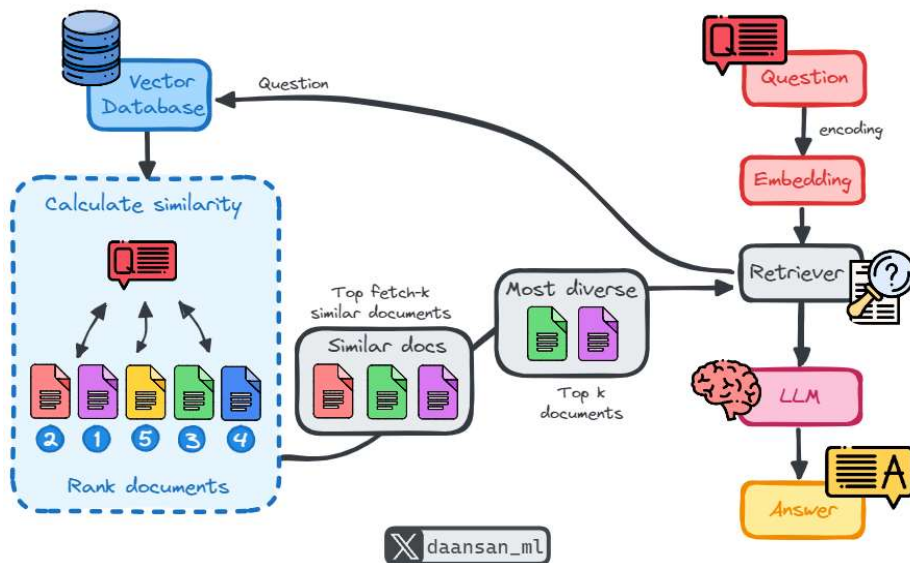
Query Routing

Semantic Routing



MMR

Maximum Marginal Relevance (MMR) Retrieval



- Maximal Marginal Relevance (MMR) is a retrieval strategy that balances relevance to the query with diversity among retrieved documents.
- Instead of selecting only the most similar vectors, MMR penalizes results that are too similar to each other.
- This prevents retrieval systems from returning redundant or near-duplicate chunks.
- MMR is especially useful in RAG systems, where repeating the same context wastes tokens and hurts generation quality.

RAG Evaluation

Faithfulness: Measures how well the answer aligns with factual information in the retrieved context.

- **Example:**

$$\text{Faithfulness} = \frac{\text{Number of generated claims present in the context}}{\text{Total number of claims made in the generated response}}$$

- Query: "What is the capital of France?"
- Context: "France is a country located in Western Europe. Paris is the most populous city in France."
- High Faithfulness Response: "Paris is the capital of France."
- Low Faithfulness Response: "The capital of France is the Eiffel Tower."

Answer Relevance: Measures how well the answer addresses the original question or prompt.

- **Example:**

$$\text{Answer Relevance} = \text{Avg} (\text{Sc} (\text{Initial Query}, \text{LLM generated Query [i]}))$$

- Query: "What are the causes of the American Civil War?"
- Context: "The American Civil War was a civil war in the United States. It was fought between the Union and the Confederacy, the latter formed by states that seceded."
- High Answer Relevance Response: "The American Civil War was fought between the northern and southern states, with slavery being a major cause."
- Low Answer Relevance Response: "The American Civil War was the deadliest conflict in American history."

$$\text{Context Relevance} = \frac{S \text{ (number of relevant sentences from the context)}}{\text{Total number of sentences in the retrieved context}}$$

Context Relevance: Measures how well the retrieved documents actually relate to the user's query.

- **Example:**

- Query: "What are the symptoms of the common cold?"
- Context 1 (High Relevance): "The common cold is a viral infection of the upper respiratory tract. Symptoms include runny nose, congestion, sore throat, and cough."
- Context 2 (Low Relevance): "The flu is a respiratory illness caused by influenza viruses. Symptoms include fever, chills, muscle aches, and fatigue."

$$\text{Context Recall} = \frac{\text{Number of Ground Truth sentences in the context}}{\text{Total number of sentences in the Ground Truth}}$$

Contextual Recall: Measures how much of the information needed for a correct answer is actually present in the retrieved documents.

- **Example:**

- Query: "Who won the first Cricket World Cup?"
- Ground Truth Answer: "England won the first Cricket World Cup in 1975."
- Context 1 (High Recall): "The Cricket World Cup was first held in England in 1975."
- Context 2 (Low Recall): "The Cricket World Cup is an international championship for cricket teams."

RAGAS

Answer Semantic Similarity: Measures how close the meaning of the generated response is to the ground truth answer, considering phrasing but not necessarily factual accuracy.

- **Example:**

- Query: "What is the tallest mountain in the world?"
- Ground Truth Answer: "Mount Everest is the world's highest mountain."
- High Semantic Similarity Response: "Everest is Earth's peak." (Uses synonyms)
- Low Semantic Similarity Response: "The Himalayas are a mountain range in Asia." (Different topic)

Answer Correctness: Measures how well the generated response matches the ground truth answer in both meaning and factual accuracy.

- **Example:**

- Query: "What is the capital of Australia?"
- Ground Truth Answer: "Canberra is the capital of Australia."
- Correct Answer: "Canberra is the capital of Australia."
- Incorrect Answer: "Sydney is the capital of Australia."

Focuses on two key areas:

- Retrieval: How well are relevant documents found for the user's query?
 - Generation: How effectively does the LLM use retrieved context to generate a response?
- **Developed by Jithin James and Shahul ES:** This framework addresses crucial evaluation questions in RAG pipelines.
 - **Evaluation Data:** Requires four components:
 - Queries (prompts) for evaluation
 - Retrieved context for each prompt
 - Corresponding LLM response/answer
 - Ground truth (known correct response)
 - **Provides Metrics for Both Retrieval and Generation:**
 - Faithfulness, Answer Relevance, Context Relevance, and Contextual Recall assess retrieval quality.
 - Answer Semantic Similarity and Answer Correctness evaluate the LLM's generated response.

Gen AI Ecosystem

Tool	Primary Role	Core Abstraction	What Problem It Solves	Typical User	Best Used When
LangChain	Orchestration framework	Chains, tools, retrievers	Helps developers connect LLMs, prompts, tools, and vector stores into pipelines	ML / Backend Engineers	You need a structured LLM pipeline (RAG, tools, simple agents)
LangGraph	Agentic workflow engine	Graphs & state machines	Manages complex, non-linear, multi-step agent behavior with memory and loops	Advanced ML / GenAI Engineers	You are building agentic systems with branching logic and state
LangSmith	Observability & evaluation	Traces, metrics, datasets	Debugging, monitoring, testing, and evaluating LLM applications	ML Engineers, Platform Teams	You want visibility, reliability, and evals in production
LangFlow	Visual builder (UI)	Drag-and-drop graphs	Rapid prototyping of LLM workflows without heavy coding	Learners, Analysts, Educators	You want fast experimentation or demos

Operational KPIs

Metric	What It Measures	Why It Matters in Production	Common Ways to Improve
Latency	Time taken to return a response	Directly affects user experience and SLA compliance	Model distillation, caching, batching, faster vector search
Throughput	Requests processed per second	Determines how many users the system can serve concurrently	Horizontal scaling, async processing, load balancing
Scalability	Ability to handle increased load	Prevents system collapse during traffic spikes	Stateless services, auto-scaling, distributed databases
Availability	Percentage of uptime	Downtime directly impacts trust and revenue	Redundancy, failover, health checks
Reliability	Consistency of system behavior	Prevents silent failures and inconsistent outputs	Monitoring, retries, circuit breakers
Cost Efficiency	Cost per request or prediction	Keeps systems economically sustainable	Model compression, prompt optimization, caching
Resource Utilization	CPU, memory, GPU usage	Prevents bottlenecks and wasted capacity	Profiling, right-sizing infrastructure

Technical Jargons

- **Semantic Search** : Semantic search refers to a search technique that goes beyond simply matching keywords to improve the relevance of search results. Here's a breakdown in two lines: Focuses on meaning: It analyzes the intent and context behind a user's query to understand the underlying meaning rather than just literal keywords.Delivers relevant results: By understanding the semantic meaning, it retrieves content that matches the user's true information need, even if the exact keywords aren't used.
- **Embeddings** : Embeddings, in the context of machine learning and natural language processing (NLP), are numerical representations of concepts or data points. Here's a breakdown in two lines:Capturing meaning: Embeddings translate concepts (like words, documents, or code) into numerical vectors, aiming to capture their semantic meaning and relationships with other concepts.Enabling analysis: These numerical representations allow machine learning models to efficiently analyze and compare concepts, facilitating tasks like information retrieval, recommendation systems, and machine translation.
- **Retriever**: In a Retrieval-Augmented Generation (RAG) system, a retriever acts as a search engine, finding relevant documents from a knowledge base based on a user query.
- **Llama-index**: LlamaIndex is a library that facilitates using large language models (LLMs) for tasks like information retrieval and question answering by creating a searchable knowledge base from your data
- **Vector Databases**: Vector databases efficiently store and retrieve high-dimensional data for tasks like semantic search and recommendation systems.